Operating System Fundamentals - EECS3221
Project 2
Design a Memory Management Unit (MMU)
Prepared by: Dr. Ruba Al Omari[1]

Contents

Instructions 1
Description 2
Specifications 2
Phase 1 – No Page Replacement 2
Address Translation 2
Handling Page Faults 3
Test File 4
How to Begin 4
Phase 2 – With Page Replacement 4
Page Replacement 4
How to Run Your Program 5
Statistics 5
How to Test Your Program 5
Deliverables 6
Rubrics 7
Grading Phase1 8
Grading Phase2 8
Collaboration 10
Generative AI Policy 10
Environment Setup 10

Instructions

- This is an individual project.
- Check eClass for the due date and time.

[1] This project is provided as a programming project in Operating System Concepts, 10th Edition, by Silberschatz et al. textbook.

Description

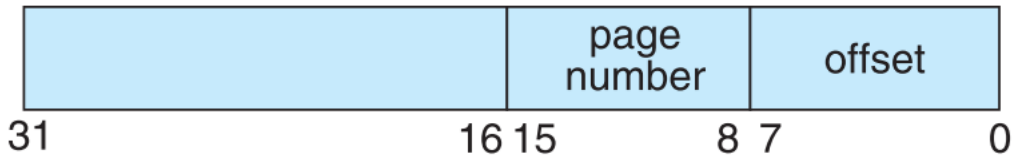
This project consists of writing a program that translates logical to physical addresses for a virtual address space of size 2^{16} = 65,536 bytes.

Your program will read from a file containing logical addresses and, using a TLB and a page table, translate each logical address to its corresponding physical address and output the value of the byte stored at the translated physical address.

Your learning goal is to use simulation to understand the steps involved in translating logical to physical addresses. This will include resolving page faults using demand paging, managing a TLB, and implementing a page-replacement algorithm.

Specifications

Your program will read a file containing several 32-bit integer numbers that represent logical addresses. However, you need to be only concerned with 16-bit addresses, so you must mask the rightmost 16 bits of each logical address. These 16 bits are divided into (1) an 8-bit page number and (2) an 8-bit page offset. Hence, the addresses are structured as shown below:



Other specifics include the following:

- 2^8 entries in the page table
- Page size of 2^8 bytes
- 16 entries in the TLB
- Frame size of 2^8 bytes
- 256 frames
- Physical memory of 65,536 bytes (256 frames × 256-byte frame size)

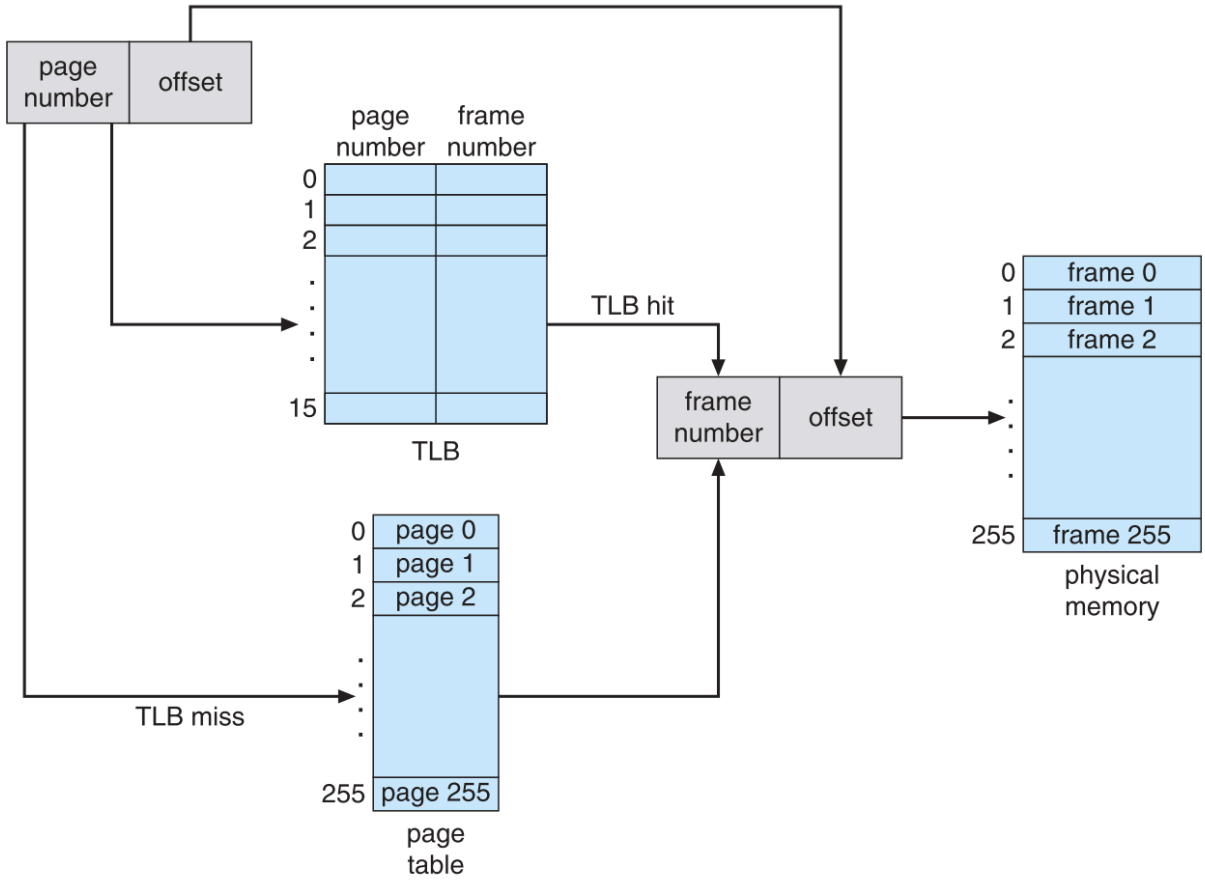
Additionally, your program needs to be only concerned with reading logical addresses and translating them to their corresponding physical addresses. You do not need to support writing to the logical address space.

Phase 1 – No Page Replacement

Address Translation

Your program will translate logical to physical addresses using a TLB and page table as outlined in Section 9.3. First, the page number is extracted from the logical address, and the TLB is consulted. In the case of a TLB hit, the frame number is obtained from the TLB. In the case of a TLB miss, the

page table must be consulted. In the latter case, either the frame number is obtained from the page table, or a page fault occurs. A visual representation of the address-translation process is:



Handling Page Faults

Your program will implement demand paging as described in Section 10.2. The backing store is represented by the file `BACKING_STORE.bin`, a binary file of size 65,536 bytes located in the `StarterKit` directory.

When a page fault occurs, you will read in a 256-byte page from the file `BACKING_STORE.bin` and store it in an available page frame in physical memory. For example, if a logical address with page number 15 resulted in a page fault, your program would read in page `15` from `BACKING_STORE.bin` (remember that pages begin at 0 and are 256 bytes in size) and store it in a page frame in physical memory. Once this frame is stored (and the page table and TLB are updated), subsequent accesses to page `15` will be resolved by either the TLB or the page table.

You will need to treat `BACKING_STORE.bin` as a random-access file so that you can randomly seek to certain positions of the file for reading. We suggest using the standard C library functions for performing I/O, including `fopen()`, `fread()`, `fseek()`, and `fclose()`.

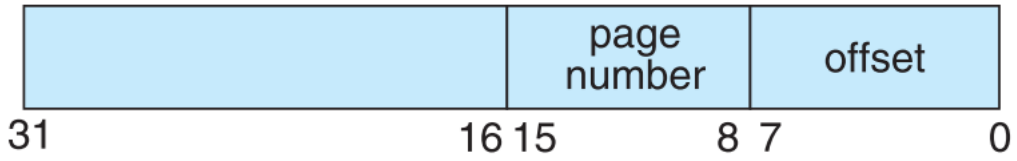
The size of physical memory is the same as the size of the virtual address space, i.e., 65,536 bytes, so you do not need to be concerned about page replacements during a page fault at this phase. Later, in phase 2, we describe a modification to this project, assuming a smaller amount of physical memory, for which a page-replacement strategy will be required.

Test File

We provide the file `addresses.txt`, which contains integer values representing logical addresses ranging from `0` to `65535` (the size of the virtual address space). Your program will open this file, read each logical address and translate it to its corresponding physical address, and output the value of the signed byte at the physical address.

How to Begin

First, write a simple program that extracts the page number and offset based on:



from the following integer numbers: `1, 256, 32768, 32769, 128, 65534, 33153`

Perhaps the easiest way to do this is by using the operators for bit-masking and bit-shifting. Once you can correctly establish the page number and offset from an integer number, you are ready to begin.

Initially, we suggest that you bypass the TLB and use only a page table. You can integrate the TLB once your page table is working properly. Remember, address translation can work without a TLB; the TLB just makes it faster. When you are ready to implement the TLB, recall that it has only 16 entries, so you will need to use a replacement strategy when you update a full TLB. A First-In-First-Out **(**FIFO**)** policy must be used for updating the TLB.
Note: The TLB replacement policy must remain FIFO in both Phase 1 and Phase 2.

Phase 2 – With Page Replacement

Page Replacement

Thus far, this project has assumed that physical memory is the same size as the virtual address space. In practice, physical memory is typically much smaller than a virtual address space. This phase of the project now assumes using a smaller physical address space with 128 page frames rather than 256. So at this phase, we have at most 2^7 valid entries in the page table (i.e., 128 pages). This change will require modifying your program so that it keeps track of free page frames as well as implementing a page-replacement policy using Least-Recently-Used **(**LRU**)** (Section 10.4) to resolve page faults when there is no free memory.

How to Run Your Program

Your program should read in the file `addresses.txt`, which contains `1,000` logical addresses ranging from `0` to `65535`. Your program is to translate each logical address to a physical address and determine the contents of the signed byte stored at the correct physical address. (Recall that in the C language, the `char` data type occupies a byte of storage, so we suggest using `char` values.)

Your program is to output a comma-separated values (CSV) file that has three columns:

1. Column 1: the logical address being translated (the integer value being read from `addresses.txt`).
2. Column 2: the corresponding physical address (what your program translates the logical address to).
3. Column 3: the signed byte value stored in physical memory at the translated physical address.

We also provide the files `correct128.csv` and `correct256.csv`, which contain the correct output values for the file `addresses.txt`. You should use this file to determine if your program is correctly translating logical to physical addresses.

Statistics

After completion, your program is to report the following statistics for both phase 1 and 2 at the end of the CSV files:

1. Page-fault rate: the percentage of address references that resulted in page faults.
2. TLB hit rate: the percentage of address references that were resolved in the TLB.

Please check the end of file `correct128.csv` and `correct256.csv` to see the correct format for reporting the statistics.

Since the logical addresses in `addresses.txt` were generated randomly and do not reflect any memory access locality, do not expect to have a high TLB hit rate.

How to Test Your Program

The file `correct256.csv` is the correct output for `addresses.txt` for phase 1 of this project. The file `correct128.csv` is the correct output for `addresses.txt` for phase 2 of this project.

You first need to complete the `Makefile` and then use `test.sh` script to test your project (i.e., you should edit the `Makefile` to reflect what files you created and how to compile them).

You don't need to perform error checking for out-of-bound values in the input file for this project. Do **not** hardcode the file names `BACKING_STORE.bin` and `addresses.txt` in your source code. These are command-line arguments that will be passed along with the physical memory size to your program, as shown in the `test.sh` file.

Note: Before running the `test.sh`, verify that your output files are not empty. An empty output file may prevent the grader from detecting errors correctly. It is your responsibility to troubleshoot and resolve any issues that affect the grader `test.sh`, including but not limited to extra blank lines, formatting differences, incorrect file names, missing files, permission problems, or other errors that may cause `test.sh` to produce unexpected results or fail to evaluate your submission correctly.

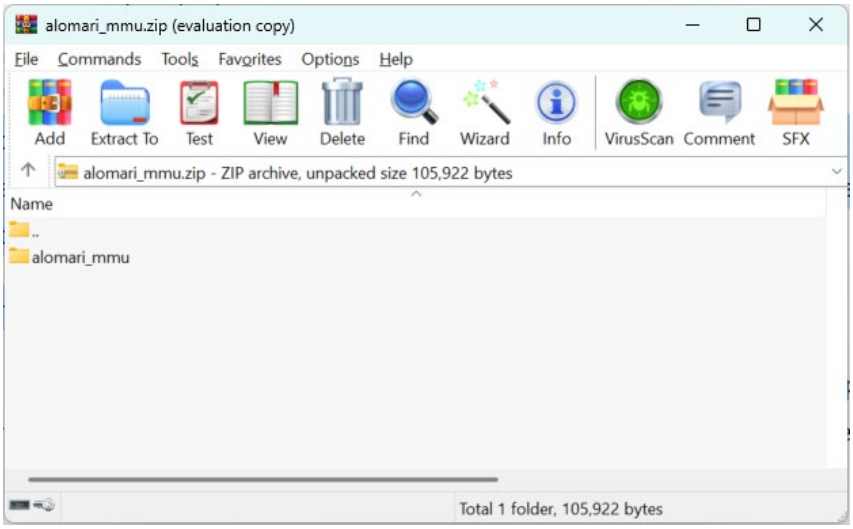
During marking, the TAs will be passing other input files that are **not** necessarily named `BACKING_STORE.bin` or `addresses.txt` to your program. Your program should work regardless of the input file names passed to it.

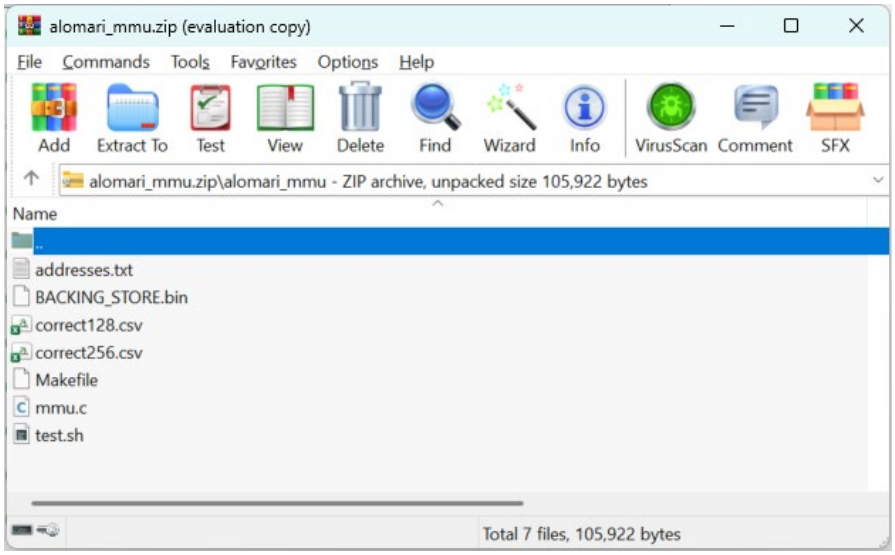
Deliverables

1- Submit a **zip** file, `yourfirstname_mmu.zip`, containing all the files that are required to build and run your project, including:
   1) `Makefile` (you may edit this file; unlike Project 1, you are expected to modify the Makefile for this project.)
   2) All C source and header files written by you.
   3) `BACKING_STORE.bin` (do not change/edit this file)
   4) `addresses.txt` (do not change/edit this file)
   5) `test.sh` (do not change/edit this file)
   6) `correct128.csv` (do not change/edit this file)
   7) `correct256.csv` (do not change/edit this file)

Please do not submit object files (*.o) or compiled executables.

All project files must be placed in a single folder. Do not create a subfolder named `StarterKit` or any other additional directory. Instead, your .zip file should contain one main folder, and inside that folder include all required files, including those originally provided in the `StarterKit`.
Below is an example of the expected structure of your zip file.





You may have more files than those shown in the screenshot. For example, I implemented the project using a single `mmu.c` file, but your implementation may be organized differently.

2- One video file that demonstrates your program working correctly:
   1- Log in to RemoteLab.
   2- Unzip your submitted yourfirstname_mmu.zip file on RemoteLab.
   3- Run test.sh showing the output for both Phase 1 and Phase 2.
   4- Explain **two** aspects of your code (see the Rubric section for more details).
   5- Your recording should be **6-8 minutes**.
   6- Your recording must be uploaded to eClass as an MP4 file (do not submit links).

Rubrics

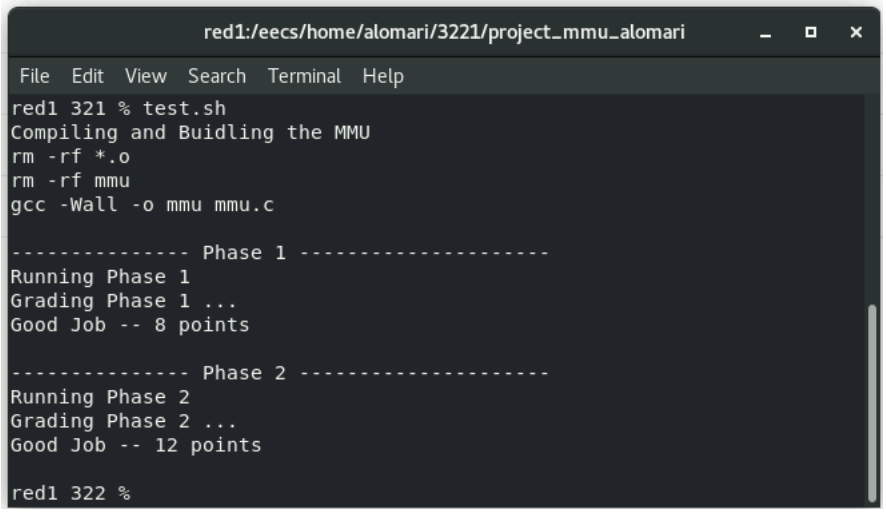
Item	**Points**
Correct `output256.csv` file for no page replacement	6
Correct statistics for no page replacement (1 point for TLB-hit rate and 1 point for page-fault rate)	2
Correct `output128.csv` file for page replacement	9
Correct statistics for page replacement (1.5 points for TLB-hit rate and 1.5 points for page-fault rate)	3
Video Demonstration	5
Sum	**25**

A. Address Translation & Page Replacement Implementation (20 points total)

The TAs will use `test.sh` bash script to grade your project. The script will be edited to use different file names in place of the `addresses.txt`, `correct128.csv`, and `correct256.csv`.

As can be seen in the `test.sh`, we first `make` your project using your submitted `Makefile`

Sample `test.sh` output:



Grading Phase1

After running make as mentioned above, `test.sh` runs:

```
./mmu 256 BACKING_STORE.bin addresses.txt
```

In which your executable, i.e., `mmu`, will be given 3 parameters: 256 as the size of the physical memory, `BACKING_STORE.bin` and `addresses.txt`, which contains the logical input addresses.

At this phase, your program should create a file called `output256.csv`, which will be compared against the `correct256.csv`.

Grading Phase2

For phase 2, we change the size of the physical memory to 128 by running:

```
./mmu 128 BACKING_STORE.bin addresses.txt
```

Here, your program should create a file called `output128.csv`, which will be compared against the `correct128.csv`.

Note: Run `test.sh` before submitting your file to ensure all is running properly. For statistics, you should set the floating-point precision to 2.

TAs will evaluate your project on one of the Department's servers with a specific configuration. You need to configure your account on one of the department servers according to this specific configuration. To do so, check the guidelines under the Environment Setup section.

B. Video Demonstration (5 points)

Record your computer screen:

1. Unzip your `yourfirstname_mmu.zip` file on RemoteLab.
2. Run `test.sh` showing the output for both phases.
3. Choose **TWO** of these aspects to explain (pick what you're most proud of or found most challenging), and open your `mmu.c` file to explain your code for each chosen aspect:

A. Data Structure Choice

- Show how you store/organize the page table, TLB, and physical memory
- Explain why you chose this structure
- Show relevant code sections

B. Algorithm-Specific Logic

- **Address Translation:** How you extract the page number and offset, and handle TLB hits/misses
- **Page Replacement (Phase 2):** How you implement LRU and track free frames
- **Statistics:** How you compute page-fault rate and TLB hit rate

C. Key Function Implementation

- Pick one key function (e.g., your address translation or page-fault handler)
- Walk through its parameters, logic, and return values
- Show how it integrates with the overall program

D. Challenge You Solved

- Describe a specific bug or challenge you encountered
- Show your debugging approach and solution
- Explain what you learned from it

What NOT to do:

- Don't explain generic paging/TLB concepts (explain YOUR implementation)
- Don't read from the project description
- Don't show your entire code file without focusing on specific parts
- Don't exceed 8 minutes

Grading Focus: We're assessing that you understand YOUR code, not that you can recite definitions.

Total: 25 points (will be scaled to 20 points for your final grade)

Collaboration

Here are the allowed activities while working on this project:

1. **Discussing Problem-Solving Approaches**: You may discuss strategies for solving a problem. However, to preserve the learning experience of others, avoid revealing key solutions directly and instead guide them toward discovering the solutions themselves.
2. **Syntax and Bug Discussions**: You can discuss syntax issues and bugs in your code, provided you do not show your code to others. For example, verbally discussing issues is allowed, but screen sharing via tools like Zoom is not.

While working on this project, it is not allowed to be:

1. **Viewing Another Student's Code**: You may not view another student's project code to understand an idea or solve a problem, nor share your own code for this purpose.
2. **Possessing Unauthorized Code**: You may not possess project solution code you didn't write yourself, nor another student's code, in any form (electronic, physical, etc.), even for debugging purposes. Sharing such code is equally prohibited.

Generative AI Policy

You may use generative AI (ChatGPT, Copilot, etc.) to explain concepts, help debug, suggest high-level structure or pseudocode, and improve documentation. You may not copy-paste complete algorithm implementations, use AI to bypass the core algorithmic work, or submit AI-generated code with only superficial changes (renamed variables, reformatting, etc.).

You must be able to explain every line of your submitted code. This is assessed through the required video demonstration; if a TA has reason to doubt your understanding from the video or your submission, you may be asked to complete a short follow-up interview, tracing through your code with a new sample input.

Submitting AI-generated code without understanding it or without properly documenting AI assistance constitutes academic dishonesty.

Environment Setup

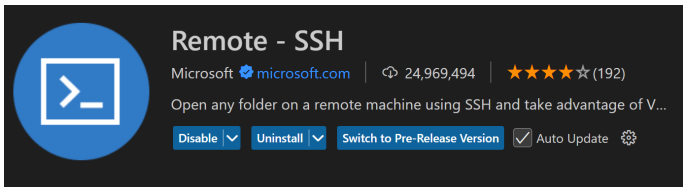
This section provides guidance for setting up EECS servers for remote development in C. You will need to configure the server and set up a client IDE (e.g., Visual Studio Code) to use the server as a backend for development.
You can use any EECS department server (e.g., red, red1, crimson, etc.) to write your code.

There are several ways you can use the servers:
Option 1: Physically visit one of the EECS labs, log in locally to the machine and start developing your code.

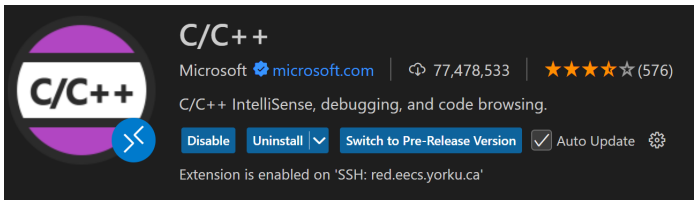
Option 2: If you can't be in the lab, you can still log in remotely to https://remotelab.eecs.yorku.ca and start developing your code.

Option 3: You can set up your local IDE to connect to the remote server following the steps below:

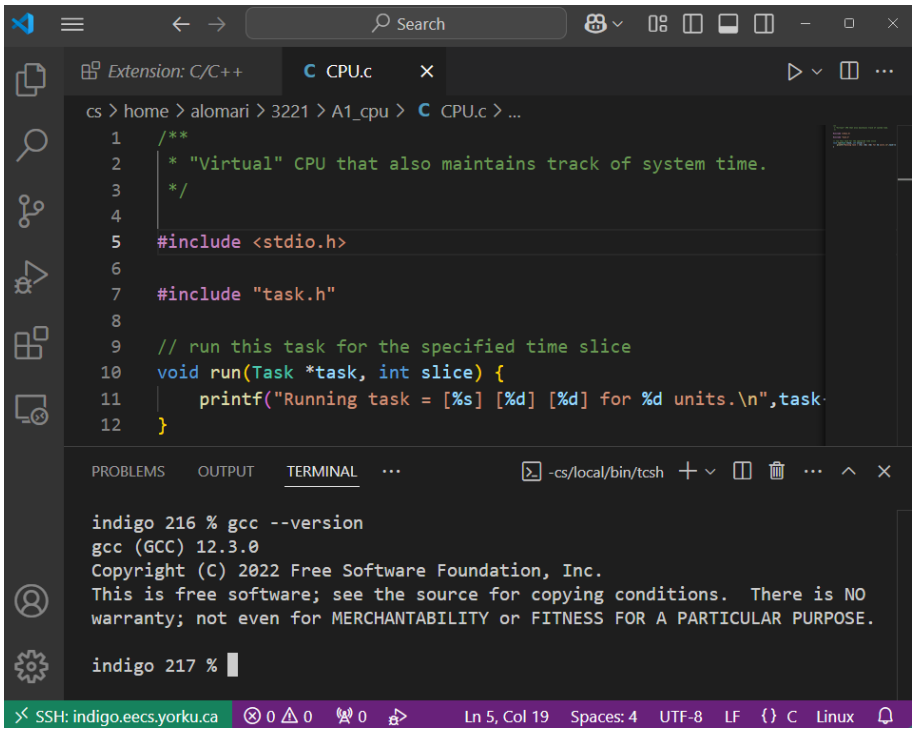
1. Download and install Visual Studio Code on your local machine.
2. Install Required Extensions:
   - Remote - SSH Extension: Allows remote connections to servers.



   - C/C++ Extension: Enables C/C++ development.



3. Create a remote connection to your, e.g., red1, server.
4. Open the terminal inside VSCode and verify the versions of GCC and make.



You are all set to start the development of your project code.

Note: GCC and other software versions on EECS servers may differ from those shown in the screenshot, as the systems are periodically updated. Your code should compile and run correctly on the current server environment.

If you encounter login issues, contact EECS IT support.